2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE) | 978-1-6654-5132-1/22/$31.00 ©2022 IEEE | DOI: 10.1109/ISSRE55969.2022.00034

A Novel Counterexample-Guided Inductive Synthesis Framework for Barrier Certificate Generation

Mi Ding, Kaipeng Lin, Wang Lin*, Zuohua Ding
School of Information Science and Technology
Zhejiang Sci-Tech University
Hangzhou, China
houge1993@foxmail.com, linkaipeng2020@163.com, linwang@zstu.edu.cn, zouhuading@hotmail.com

***Abstract*—Barrier certificate is a powerful and practical approach of safety verification for hybrid systems. In this paper, we propose a novel Counterexample-Guided Inductive Synthesis (CEGIS) procedure for synthesizing neural barrier certificates. The CEGIS procedure is structured as an inductive loop where a *learner* and a *verifier* interact to synthesize barrier certificates. The *learner* trains candidate barrier certificates expressed as feedforward neural networks with polynomial activations, and the *verifier* employs computer algebra techniques to either ensure the validity of the trained candidate barrier certificate or produce informative counterexamples, which can effectively reduce the number of CEGIS iterations. We implement the CEGIS tool and evaluate its performance over a set of benchmarks. The experimental results demonstrate the effectiveness and efficiency of our approach.**

***Index Terms*—Barrier certificate, counterexample-guided inductive synthesis, neural networks, computer algebra techniques**

I. Introduction

Much research effort has been devoted to safety verification of hybrid systems for the reliability requirement of safety-critical cyber physical systems (CPS), such as drones, self-driving cars, and satellite rendezvous. Safety verification contributes to checking safety properties by determining whether a system can evolve to some states violating desired safety properties. A successful verification can give more confidence in the verified systems.

Due to their ability to handle nonlinear and uncertain systems for infinite time horizon, barrier certificate generation based methods have been well-studied in solving the safety verification problem of hybrid systems. A barrier certificate is a function whose zero level set separates the unsafe set from all reachable states of a system. As the existence of barrier certificates implies that the unsafe set is not reachable, the problem of safety verification can be transformed into the problem of barrier certificate generation.

This work was partially supported by the Zhejiang Provincial Key Research and Development Program of China under Grant 2022C01045, the Zhejiang Provincial Natural Science Foundation of China under Grant LY20F020020, and the National Natural Science Foundation of China under Grant 62132014.
*Wang Lin is the corresponding author.

Barrier certificate generation is a compute intensive task. Symbolic computation based methods [1], [2], such as quantifier elimination or Grönber bases computation, have been applied to solve the quantification problem which can prove the existence of barrier certificates. However, symbolic methods suffer from the scalability problem.

From a computational point of view, convex relaxation based methods provide much better efficiency at the cost of more conservative results. Instead of directly handling constraints with quantifiers, sums of squares (SOS) relaxation or linear relaxation transforms them to more conservative constraints represented as linear (matrix) inequalities or bilinear (matrix) inequalities, and then solves it quickly via numerical optimization methods such as semidefinite programming (SDP), linear programming (LP) and interval analysis [3]–[6]. However, there is a trade-off between the expressiveness of the barrier certificate conditions and the efficiency in discharging the reduced constraints.

Furthermore, deep neural networks have the potential to define more complex barrier certificates, since that feedforward neural networks have the universal approximation property [7]. The seminal work introducing neural networks as barrier certificates is proposed by Zhao et al. [8]. They exploit neural networks with Bent-ReLU activation functions to verify dynamical systems. In [9], Zhao et al. adopt a special feedforward neural network structure with two hidden layers and use the ReLU activation functions to synthesize candidate barrier certificates, and employ a MILP optimization based technique to identify the real ones.

Peruffo et al. [10], [11] present a CounterExample-Guided Inductive Synthesis (CEGIS) procedure to speed up the construction of networks. The procedure is structured as an inductive loop between a learner, which trains a neural network to fit over a finite set of samples the requirements for a barrier certificate, and a sound verifier, which either certifies the candidate's validity via an SMT solver or generates counterexamples to further guide the learner. For CEGIS technique, its effectiveness and practicality are decided to a large extent by the scalability of checking satisfiability and the quality of counterexamples, therefore the method for effective satisfi-

2332-6549/22/$31.00 ©2022 IEEE
DOI 10.1109/ISSRE55969.2022.00034

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

ability checking and informative counterexample generation becomes a key point.

In this paper, following Peruffo et al.'s line, we propose a novel CEGIS procedure for synthesizing barrier certificates represented by deep neural networks for safety verification of hybrid systems. The *learner* in our CEGIS procedure trains candidate barrier certificates expressed as feedforward neural networks with polynomial activations approximated from the typical activation functions (such as ReLU, sigmoid, and tanh functions). The approximate polynomials inherit intrinsic properties from the typical activations to some extent, thus facilitates the training of neural barrier certificates. The *verifier* employs a Cylindrical Algebraic Decomposition (CAD) solver to either certify the validity of the candidates, or produce multiple informative counterexamples at a time. Moreover, it can generate an enhanced worst counterexample to each barrier condition via SOS programming, to speed up synthesizing a valid barrier certificate.

The main contributions of this paper are summarized as follows:

- We introduce a learning-based approach for synthesizing polynomial barrier certificates via neural network training and verification. It leverages efficient supervised learning algorithm, while guaranteeing formal soundness via CAD based solver.
- Our CEGIS procedure employs two strategies to search for informative counterexamples to barrier conditions from different perspectives, which can effectively reduce the number of CEGIS iterations.
- We implement the tool *CEGIS4BC* and provide a detailed experimental evaluation on a set of benchmarks, which shows that our approach is more effective than the existing method.

The structure of this paper is as follows. Section 2 briefly introduces some preliminary knowledge. In Section 3, we describe details about training and verification of neural barrier certificates corresponding to Algorithm 1. Section 4 provides an experimental evaluation of our algorithm with several examples from relative literature. We illustrate the related work in Section 5 before the conclusion in Section 6.

II. Preliminaries

This section introduces the definitions used in this paper and notations in the rest of the paper. Let $\mathbb{R}$ be the field of real number. $\mathbb{R}[\mathbf{x}]$ denotes the polynomial ring with coefficients in $\mathbb{R}$ over variables $\mathbf{x} = [x_1, x_2, \cdots, x_n]^T$, and $\Sigma[\mathbf{x}] \subset \mathbb{R}[\mathbf{x}]$ denotes the space of SOS polynomials.

A. Safety Verification with Barrier Certificates

Hybrid systems involve both continuous dynamics as well as discrete transitions. To model hybrid systems, we use the notion of hybrid automata [12].

Definition 1 (Hybrid Automata): A *hybrid automaton* is a system $\mathbf{H} : \langle L, X, F, \Psi, E, G, R, \Theta, \ell_0 \rangle$, where

- L, a finite set of locations (or models);
- $X \subseteq \mathbb{R}^n$ is the continuous state space. The hybrid state space of the system is defined by $\mathcal{X} = L \times X$ and a state is defined by $(\ell, \mathbf{x}) \in \mathcal{X}$;
- $F : L \to (\mathbb{R}^n \to \mathbb{R}^n)$, assigns to each location $\ell \in L$ a locally Lipschitz continuous vector field $\mathbf{f}_\ell$;
- Ψ assigns to each location $\ell \in L$ a *location condition (location invariant)* $\Psi(\ell) \subseteq \mathbb{R}^n$;
- $E \subseteq L \times L$ is a finite set of discrete transitions;
- G assigns to each transition $e \in E$ a switching guard $G_e \subseteq \mathbb{R}^n$;
- R assigns to each transition $e \in E$ a reset function $R_e : \mathbb{R}^n \to \mathbb{R}^n$;
- $\Theta \subseteq \mathbb{R}^n$, an initial continuous state set;
- $\ell_0 \in L$, the initial location. The initial state space of the system is defined by $\ell_0 \times \Theta$.

A trajectory [13] of $\mathbf{H}$ is an infinite sequence of states

$$(l_0, \mathbf{x}_0),\ (l_1, \mathbf{x}_1), \cdots, (l_i, \mathbf{x}_i),\ (l_{i+1}, \mathbf{x}_{i+1}), \cdots$$

such that

- **[Initiation]** $(l_0, \mathbf{x}_0) \in \ell_0 \times \Theta$;
  Furthermore, for each consecutive pair $(l_i, \mathbf{x}_i), (l_{i+1}, \mathbf{x}_{i+1})$, one of the two *consecution* conditions holds:
- **[Discrete Consecution]** $e = (l_i, l_{i+1}) \in E$, $\mathbf{x}_i \in G_e$ and $x_{i+1} = R_e(\mathbf{x}_i)$; or
- **[Continuous Consecution]** $l_i = l_{i+1} = \ell$, and there exists a time interval $\delta > 0$ such that the solution $\mathbf{x}(\mathbf{x}_i; t)$ to $\dot{\mathbf{x}} = \mathbf{f}_\ell$ evolves from $\mathbf{x}_i$ to $\mathbf{x}_{i+1}$, while satisfying the location invariant $\Psi(\ell)$. Formally,
  - $\mathbf{x}(\mathbf{x}_i, \delta) = \mathbf{x}_{i+1}$ and
  - $\forall t \in [0, \delta], \mathbf{x}(\mathbf{x}_i, t) \in \Psi(\ell)$.

A state $(\ell, \mathbf{x})$ is called a *reachable state* of a hybrid system $\mathbf{H}$ from the initial state set $\ell_0 \times \Theta$ if it appears in some trajectory of $\mathbf{H}$. During a continuous flow, the discrete location ℓ_i is maintained and the continuous state variables $\mathbf{x}$ evolve according to the differential equations $\dot{\mathbf{x}} = \mathbf{f}_{\ell_i}(\mathbf{x})$, with $\mathbf{x}$ satisfying the location invariant $\Psi(\ell_i)$. At the state $(\ell_i, \mathbf{x})$, if there is a discrete transition $e = (\ell_i, \ell_j) \in E$ such that $\mathbf{x} \in G_e$, the system may undergo a transition to location ℓ_j, and $\mathbf{x}$ will take the new value $\mathbf{x}'$, which is determined by the reset function R_e.

Given a hybrid system $\mathbf{H}$ with prespecified unsafe state set $\mathcal{X}_u \subseteq \mathcal{X}$, we say that system $\mathbf{H}$ is *safe* if all trajectories of $\mathbf{H}$ starting from the initial state set $\ell_0 \times \Theta$, can not evolve to any state specified by $\mathcal{X}_u$. Therefore, the problem of verifying the safety property is to decide whether $\mathbf{H}$ is safe, or, any state specified by $\mathcal{X}_u$ is not reachable.

For safety verification of hybrid systems, the concept of barrier certificates [14] plays an important role. A barrier certificate maps all the states in the reachable set to non-negative reals and all the states in the unsafe set to negative reals. Clearly, the zero level set of a barrier certificate acts a barrier to separate the reachable set from the unsafe set. The following theorem states the conditions that must be satisfied by a barrier certificate.

Theorem 1: Given a hybrid system $\mathbf{H}$ with prespecified unsafe state set $\mathcal{X}_u \subseteq \mathcal{X}$. Suppose there exists a differentiable

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

function $B_\ell(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$ for each location $\ell \in L$ satisfying the following conditions:

(i) $B_{\ell_0}(\mathbf{x}) \geq 0 \quad \forall \mathbf{x} \in \Theta$,

(ii) $B_\ell(\mathbf{x}) = 0 \Rightarrow \mathcal{L}_f B_\ell(\mathbf{x}) > 0 \quad \forall \mathbf{x} \in \Psi(\ell)$,

(iii) $B_\ell(\mathbf{x}) \geq 0 \Rightarrow B_{\ell'}(R_e(\mathbf{x})) \geq 0 \quad \forall e = (\ell, \ell') \in E \forall \mathbf{x} \in G_e$,

(iv) $B_\ell(\mathbf{x}) < 0 \quad \forall \mathbf{x} \in X_u(\ell)$,

where $\mathcal{L}_f B_\ell(\mathbf{x})$ denotes the Lie-derivative of $B_\ell(\mathbf{x})$ along the vector field $\mathbf{f}_\ell(\mathbf{x})$, i.e., $\mathcal{L}_f B_\ell(\mathbf{x}) = (\nabla B_\ell \cdot \mathbf{f}_\ell(\mathbf{x})) = \sum_{i=1}^{n} \frac{\partial B_\ell}{\partial x_i} \cdot f_{\ell i}(\mathbf{x})$, then $\{B_\ell(\mathbf{x})\}$ is a barrier certificate, and the safety of system $\mathbf{H}$ is guaranteed.

In this paper, we focus on semi-algebraic hybrid systems whose elements are represented as polynomial relations (equalities and inequalities) over the system variables. In what follows, the definition of semi-algebraic hybrid system is provided.

Definition 2: [Semi-algebraic Hybrid System] A semi-algebraic hybrid system is a hybrid system: $\mathbf{H} : \langle L, X, F, \Psi, E, G, R, \Theta, \ell_0 \rangle$, where

- the continuous vector field $F(\ell)$ for each $\ell \in L$ is of the form $\dot{\mathbf{x}} = \mathbf{f}_\ell(\mathbf{x})$, where $\mathbf{f}_\ell(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]^n$;
- the initial condition Θ, the location invariant $\Psi(\ell)$ for each $\ell \in L$, and the guard condition G_e for each $e \in E$ are semi-algebraic sets defined by polynomial inequalities with variables $\mathbf{x}$; $R_e \in \mathbb{R}[\mathbf{x}]^n$ is the reset function for each $e \in E$.

For ease of presentation, the semi-algebraic sets Θ, $\Psi(\ell)$ and G_e in Definition 2 are represented as follows:

$$\begin{aligned} \Theta : &= \{\mathbf{x} \in \mathbb{R}^n \mid \theta_1(\mathbf{x}) \geq 0, \ldots, \theta_q(\mathbf{x}) \geq 0\}, \\ \Psi(\ell) : &= \{\mathbf{x} \in \mathbb{R}^n \mid \psi_{\ell,1}(\mathbf{x}) \geq 0, \ldots, \psi_{\ell,r}(\mathbf{x}) \geq 0\}, \\ G_e : &= \{\mathbf{x} \in \mathbb{R}^n \mid g_{e,1}(\mathbf{x}) \geq 0, \ldots, g_{e,s}(\mathbf{x}) \geq 0\}, \end{aligned}$$

where $\ell \in L$, $e \in E$, and $\theta_i(\mathbf{x})$, $\psi_{\ell,j}(\mathbf{x})$, $g_{e,k}(\mathbf{x})$ are polynomials. Furthermore, we also assume that the unsafe set $\mathcal{X}_u$ is a semi-algebraic set, defined by

$$X_u(\ell) := \{\mathbf{x} \in \mathbb{R}^n \mid \zeta_{\ell,1}(\mathbf{x}) \geq 0, \ldots, \zeta_{\ell,p}(\mathbf{x}) \geq 0\},$$

where $\zeta_{\ell,i} \in \mathbb{R}[\mathbf{x}], 1 \leq i \leq p$. In addition, hereafter we assume that the above semi-algebraic sets are compact.

Note that an extension of our approach to more general nonlinear hybrid systems involving transcendental functions will be demonstrated in subsequent sections.

B. Neural networks

A general feed-forward neural network consists of a number of interconnected neurons that are organized as a layered structure, including an input layer, an output layer, and several hidden layers between the first two ones. The neurons in each layer receive signals from the neurons in preceding layer and output the processed ones to the neurons of the next layer. We give some notions to describe the network. Subscript 0 and L are reserved for indices of the input layer and output layer, respectively. Subscript l is used to index layer $l-$specific variables. W_l is the weight between layer $l-1$ to layer l and $\mathbf{b}_l$ is the bias of layer l. We define the activation function of layer l as σ_l.

The network is fed an input through its input layer, which is then propagated through the layers by successive application of linear calculations and activation functions until it reaches the output layer. Let $\mathbf{x}_0$ denote the given value of neurons in the first layer, $\mathbf{z}_l$ and $\mathbf{x}_l$, $1 \leq l \leq L$ denote the neuron value of hidden layer l before and after activation function σ_l, respectively, and $\mathbf{x}_L$ is the neuron value of the output layer. The following equations describe how the forward propagation of deep neural network conducts from input layer to output layer:

$$\begin{cases} \mathbf{z}_l = W_l \mathbf{x}_{l-1} + \mathbf{b}_l & l = 1, 2, ..., L-1 \\ \mathbf{x}_l = \sigma_l(\mathbf{z}_l) & l = 1, 2, ..., L-1 \\ \mathbf{x}_L = W_L \mathbf{x}_{L-1} + \mathbf{b}_L \end{cases} \tag{1}$$

Intuitively, a feed-forward neural network defines a function B and can be represented as a composition of its layers. Therefore, we have $B(\mathbf{x}_0) = \mathbf{x}_L$, which means that the network output $B(\mathbf{x}_0)$ is the neuron value of the output layer. This paper aims to synthesize barrier certificates in the form of neural networks.

III. Synthesis of Barrier Certificate via Learning and Verification

In this section, we introduce a novel CounterExample-Guided Inductive Synthesis (CEGIS) procedure for learning barrier certificates represented by deep neural networks, satisfying the barrier conditions in Theorem 1. The CEGIS procedure is structured as an inductive loop where a *learner* and a *verifier* interact to synthesize barrier certificates.

More precisely, the *learner* trains a candidate neural barrier certificate that satisfies the barrier conditions in Theorem 1 over a set of sampled data, by encoding the barrier conditions into the loss function of the neural network, until the loss stabilizes at zero. For the trained neural barrier certificate, the *verifier* either ensures its validity via an CAD based solver or yields the enhanced worst counterexample with respect to each barrier condition via SOS programming. The counterexamples are passed back to further guide the *learner* to refine the candidate.

A. Training of neural barrier certificates

In the following, we focus on the *learner* component and show how to train neural barrier certificates using supervised learning. The feedforward neural networks of a specific structure are employed as the templates for barrier certificates, which consist of an input layer, an output layer, and $L-1$ hidden layers. Especially, the number of input neurons is equal to the dimension of the state $\mathbf{x}$ in the system (1), and the number of output neurons is 1, which means that the output of the neural network is a scalar value.

Activation Function. In [11], a γ-order polynomial activation: $\sigma(x) = x^\gamma$ is adopted, due to the need for interpretable outputs from the neural network, whose analytical expression can be readily processed by the verifier. In this paper, we also adopt polynomials as the legal activation functions. Exploiting the

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

fact that the neural networks with nonlinear activation functions have the universal approximation property, we choose polynomial approximations of the typical activations, such as rectified linear unit (*ReLU*), sigmoid, and tanh functions.

Practically speaking, consider a typical activation $\sigma(x)$ with $x \in [a, b]$, where $[a, b]$ is an interval representing the bound of the neuron, its d-th order polynomial with trainable weights w_j, $j = 0, 1, \cdots, d$ is defined as $f_d(x) = \sum_{j=0}^{d} w_j \cdot x^j$. It is reasonable to choose w_j such that $f_d(x) \approx \sigma(x)$. To this end, we fit a polynomial of degree d to $m > d$ with data points $(x_i, \sigma(x_i))$, $i = 1, \cdots, m$ with $x_i \in [a, b]$, using the least squares approach, i.e., to find w_j such that

$$\sum_{i=1}^{m} [\sigma(x_i) - f_d(x_i)]^2$$

is minimized.

Note that the polynomial activation $f_d(x)$ can be adaptively computed. Let $D^{[0]}$ be the interval vector representing the bounds of the input neurons, and $D^{[l-1]}$, $1 \leq l \leq L$ be the interval vector representing the bounds of the $l-1$-th layer neurons. The bounds of all neurons can be obtained by performing the procedure of the layer-by-layer analysis. More specifically, suppose $D^{[l-1]}$ is yielded from the previous layer, applying the interval computation to (1) yields the range of $\mathbf{z}_j^{[l]}$, denoted by $[\alpha_{l,j}, \beta_{l,j}]$ for each $j = 1, \cdots, s_l$ where s_l is the number of neurons in the l-th layer. Since σ is a monotonically increasing function, the i-th element of $\mathbf{x}^{[l]}$ can be bounded by $\sigma(\alpha_{l,i}) \leq \mathbf{x}_i^{[l]} \leq \sigma(\beta_{l,i})$. As a result, we have $\mathbf{x}^{[l]} \subseteq D^{[l]}$, where $D^{[l]} = [\sigma(\alpha_{l,1}), \sigma(\beta_{l,1})] \times \cdots \times [\sigma(\alpha_{l,s_l}), \sigma(\beta_{l,s_l})]$.

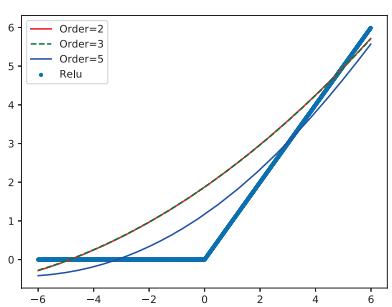

Fig. 1. ReLU approximations derived for polynomial activations.

Theoretically, any approximate polynomials can be taken as the legal activation functions. To ease computation, one prefers to setting them as simple as possible. Here we choose approximate polynomials of degrees 2, 3 and 5. Figure 1 shows the approximations of ReLU functions using such polynomials. Note that neural networks are the combination of linear calculations and activation functions. It can be easily concluded that feed-forward neural networks with polynomial activation functions are also polynomials, which are continuously differentiable as required by barrier certificates.

Training data generation. In the following, we show how to generate sufficient data sets for the *learner* to train candidate neural barrier certificates. The datasets are generated for pushing the neural network to fit the barrier conditions given in Theorem 1. Now we illustrate the sampling process for one network $B_\ell(\mathbf{x})$ of location $\ell, \ell \in L$ and the others' can be finished in the same manner.

Intuitively, four datasets are to be generated corresponding to four barrier conditions in Theorem 1. There is one initial dataset for condition (i), one domain dataset corresponding to location $\ell \in L$ for condition (ii), one transition dataset from location ℓ' to location ℓ for condition (iii), and one unsafe dataset in location ℓ for condition (iv). Let $\mathcal{D}_1$ and $\mathcal{D}_{\ell,2}$, $\mathcal{D}_{\ell,3}, \ell \in L$ denote the subset in which contains the data sampled from the initial set Θ, the domain $\Psi(\ell)$ for each location, and respective transition states G_e, respectively. When fed with the points in $\mathcal{D}_1$ or $\mathcal{D}_{\ell,2}$, the network $B_\ell(\mathbf{x})$ is supposed to output non-negative values. It is worth noting that the network $B_\ell(\mathbf{x})$ should also be non-negative with the constraint about $B_{\ell'}(\mathbf{x}') \geq 0$ on the feeding of $\mathcal{D}_{\ell,3}$. On the contrary, the network should generate negative value with the feeding of points in $\mathcal{D}_{\ell,4}$ sampled from the unsafe set $X_u(\ell)$.

The approach of mesh generation is employed for sampling. Here, we introduce the process for the generation of $\mathcal{D}_1$. Let $\mathbf{l}, \mathbf{u} \in \mathbb{R}^n$ be the vectors of the lower bound and upper bound of Θ, i.e., $\Theta \subseteq [\mathbf{l}, \mathbf{u}]$. First of all, we sample from each dimension of $[\mathbf{l}, \mathbf{u}]$ equidistantly with a fixed mesh space δ. The construction of training data from Θ has been accomplished when the region portrayed by $[\mathbf{l}, \mathbf{u}]$ approximately equal to the initial region. Otherwise, it is necessary to construct a filter for excluding the states that dissatisfy the constraints for initial region. The dataset $\mathcal{D}_{\ell,3}$ and $\mathcal{D}_{\ell,4}$ can be constructed in the same manner. Remark that the generation of $\mathcal{D}_{\ell,2}$ is slightly different from $\mathcal{D}_1$. As mentioned before, dataset $\mathcal{D}_{\ell,2}$ comes from the domain in location ℓ. Assuming we get a subset of the domain in the same manner with dataset $\mathcal{D}_1$ called $\mathcal{D}_\ell$. In the second condition of Theorem 1, the lie derivative of the barrier certificate is supposed to be non-negative when $B_\ell(\mathbf{x}) = 0$. Consequently, we filter $\mathcal{D}_\ell$ with $B_\ell(\mathbf{x}) = 0$ to get dataset $\mathcal{D}_{\ell,2}$. In practice, it is difficult to sample points on the 0-level set of $B_\ell(\mathbf{x})$. Thus we give a relaxed condition that a gap is set around the boundary $B_\ell(\mathbf{x})$. That is, given a constant τ (usually 0.1), when $B_\ell(\mathbf{x}) \leq |\tau|, \mathcal{L}_f B_\ell(\mathbf{x}) \geq 0$.

Note that the cardinalities of the datasets grow exponentially in the dimension of the state space, which in turn makes the training of neural barrier certificates highly time-consuming. A potential way to overcome the complexity bottleneck is to discretize different state sets with a heterogeneous instead of a uniform granularity.

Loss function. The learning process updates the network parameters to improve the likelihood of satisfying the barrier conditions in Theorem 1 over the datasets $\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}$, and $\mathcal{D}_{\ell,4}$ which are encoded as a loss function, named the barrier

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

loss. More precisely, the barrier loss is defined as follows:

$$\mathcal{L}_{total} = \sum_{\mathbf{x}\in\mathcal{D}_1} \max(-B_\ell(\mathbf{x}) + \epsilon_1, 0) + \sum_{\mathbf{x}\in\mathcal{D}_{\ell,3}} \max(-B_\ell(\mathbf{x}) + \epsilon_3, 0) + \sum_{\mathbf{x}\in\mathcal{D}_{\ell,2}} \max(-\mathcal{L}_f B_\ell(\mathbf{x}) + \epsilon_2, 0) + \sum_{\mathbf{x}\in\mathcal{D}_{\ell,4}} \max(B_\ell(\mathbf{x}) + \epsilon_4, 0)$$

where ϵ_1, ϵ_2, ϵ_3 and ϵ_4 are small non-negative constants for fine tuning. The terms in barrier loss are directly linked to condition $(i - iv)$ in Theorem 1, which measures the degree of violation of the barrier conditions. The first, second and third terms are designed to force the network output $B_\ell(\mathbf{x})$ to have non-negative value on related training set by penalizing the points with $B_\ell(\mathbf{x}) < 0$, while the fourth term guarantees the strict negative value for the network output $B_\ell(\mathbf{x})$. We minimize $\mathcal{L}_{total}$ to update neural parameters by backward propagation for training a candidate barrier certificate. Furthermore, an example is used to depict how our synthesis method works.

Example 1: Consider the following polynomial dynamical system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} -x_2 \\ -x_3 \\ -x_1 - 2x_2 - x_3 + x_1^3 \end{bmatrix}$$

with domain $\mathcal{X} = \left\{\mathbf{x} \in \mathbb{R}^3 | -3 \leq x_1, x_2, x_3 \leq 3\right\}$. Our goal is to identify whether the above system is safe by checking that all trajectories of the system starting from the initial set

$$\mathcal{X}_0 = \left\{\mathbf{x} \in \mathbb{R}^3 | (x_1 - 0.5)^2 + (x_2 - 0.5)^2 + (x_2 + 0.5)^2 \leq 0.25\right\}$$

will never reach the unsafe set

$$\mathcal{X}_u = \left\{\mathbf{x} \in \mathbb{R}^3 | (x_1 - 1.75)^2 + (x_2 + 1.75)^2 + (x_2 + 1.75)^2 \leq 0.16\right\}.$$

For the fact that barrier certificate can act as a barrier which separates the safe states and unsafe states of the system, we can transform the above problem into synthesizing a real barrier certificate. The neural barrier certificate is first parameterized as a single-layer neural network of 30 neurons, with polynomial activation of degree 2. Network parameters are then updated using Adam optimizer with learning rate of 0.05. The empirical barrier loss $\mathcal{L}_{total}$ decreases quickly over iterations, and converges to zero after 5000 times of parameter updates. The learning procedure successfully outputs a candidate neural barrier certificate

$$\begin{aligned}\tilde{B}(\mathbf{x}) =& 1.1211x_1^2 + 0.0368x_1x_2 - 1.0597x_1x_3 - 0.4984x_1 \\ &+ 0.2038x_2^2 + 0.2426x_2x_3 - 0.3909x_2 + 0.1605x_3^2 \\ &+ 0.1322x_3 - 1.5575.\end{aligned}$$

□

B. Verification of neural barrier certificates

The learning stage cannot provide a formal guarantee for the candidate barrier certificate, which means the verification is necessary. For semi-algebraic hybrid systems, verification of candidate barrier certificates can be achieved by solving satisfiability of the corresponding polynomial formulas:

$$\begin{cases} \forall \mathbf{x} \in \Theta. \qquad B_{\ell_0}(\mathbf{x}) \geq 0 \\ \forall \mathbf{x} \in \Psi(\ell). \qquad B_\ell(\mathbf{x}) = 0 \Rightarrow \mathcal{L}_f B_\ell(\mathbf{x}) > 0 \\ \forall e = (\ell, \ell') \in E \forall \mathbf{x} \in G_e.\ B_\ell(\mathbf{x}) \geq 0 \Rightarrow B_{\ell'}(R_e(\mathbf{x})) \geq 0 \\ \forall \mathbf{x} \in X_u(\ell). \qquad B_\ell(\mathbf{x}) < 0, \end{cases} \tag{2}$$

which belong to the decidable theory of real closed fields. The most popular algorithm for deciding sentences over algebraic expressions is Cylindrical Algebraic Decomposition (CAD), and several CAD based solvers, such as Mathematica and QEPCAD, can provide formal result for the satisfiability checking.

More specifically, REDUCE function in Mathematica can be utilized for the unsatisfiability checking of the barrier conditions, and FINDINSTANCE function can be used for generating counterexamples. Note that CAD algorithm computes a cell decomposition of any semi-algebraic set presented by a quantified system of polynomial equations and inequalities, and yields a sample point in each of the cells. Therefore, CAD based solver can yield multiple counterexamples to each barrier condition at a time, thus effectively reduces the number of iterations and speeds up synthesizing a valid barrier certificate.

Furthermore, CAD based solvers also can be used to verify the validity of candidate barrier certificates for nonlinear hybrid systems involving transcendental functions (such as exponential functions, logarithms and trigonometric functions). For example, the REDUCE command is capable of solving decision problems for systems of equations and inequalities involving real univariate exp-log functions or tame elementary functions.

Remark 1: The CAD method has the worst case with complexity doubly exponential in the number of variables, which remains a serious limit to its scalability. Fortunately, there exist numerous approaches used to improve the efficiency and scalability of CAD method, such as improvements to the projection operator [15], partial CAD [16], and symbolic-numeric lifting schemes [17].

In CEGIS procedure, the choice of counterexamples is the key to the performance in refinement of the candidate barrier certificate. Therefore, informative counterexamples can be utilized to guide the *learner* to speed up synthesizing a valid barrier certificate. In the following, we propose a strategy to search for the most suspected point, which is the most likely to violate each barrier condition in Theorem 1. In fact, all barrier conditions can be expressed as a unified type, say, a polynomial is nonnegative (positive) on a semi-algebraic set. Consider the initial barrier condition: $\forall \mathbf{x} \in \Theta, B_\ell(\mathbf{x}) \geq 0$ with $\Theta := \{\mathbf{x} \in \mathbb{R}^n \,|\, p_j(\mathbf{x}) \geq 0, j = 1, \ldots, m_2\}$, one can construct the following polynomial optimization problem:

$$\left.\begin{aligned} B^*_{POP} &= \min_{\mathbf{x}} B_\ell(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{x} \in \Theta, \end{aligned}\right\} \tag{3}$$

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

which computes the infimum B^*_{POP} of the polynomial $B_\ell(\mathbf{x})$ over the semi-algebraic set Θ. Clearly, the initial barrier condition holds if and only if $B^*_{POP} \geq 0$. Therefore, the optimal solution $\mathbf{x}^*_{POP}$ to problem (3) is the point that is the most likely to violate the initial barrier condition. Especially, if the optimal solution $\mathbf{x}^*_{POP}$ was such that $B^*_{POP} < 0$, the $\mathbf{x}^*_{POP}$ was the worst counterexample of the initial barrier condition.

Generally, the problem (3) is a hard nonlinear and non-convex optimization problem. Indeed, finding infimum B^*_{POP} amounts to finding the largest scalar λ for which the polynomial $B - \lambda$ is nonnegative on the semi-algebraic set Θ. Therefore, the problem (3) can be rewritten as the following equivalent optimization problem

$$\left.\begin{array}{ll} B^*_{POP} = & \max\limits_{\mathbf{x}} \ \lambda \\ \text{s.t.} & B_\ell(\mathbf{x}) - \lambda \geq 0, \ \forall \mathbf{x} \in \Theta. \end{array}\right\} \tag{4}$$

As is shown in [18], Putinar's Positivstellensatz provides a powerful representation for polynomial positivity on a semi-algebraic set. Let $\mathcal{K} = \{\mathbf{x} \in \mathbb{R}^n \mid g_1(\mathbf{x}) \geq 0, \ldots, g_s(\mathbf{x}) \geq 0\}$ be a basic semi-algebraic set, and $\mathbf{g} = \{g_1(\mathbf{x}), \ldots, g_s(\mathbf{x})\}$ where $g_j \in \mathbb{R}[\mathbf{x}], 1 \leq j \leq s$, the polynomial set defined by

$$M(\mathbf{g}) := \{\sigma_0 + \sum_{i=1}^{s} \sigma_i g_i \mid \sigma_i \in \Sigma[\mathbf{x}], \, 0 \leq i \leq s\}$$

is called the quadratic module generated by $\mathbf{g}$.

Theorem 2: [Putinar's Positivstellensatz] Assume that the quadratic module $M(\mathbf{g})$ is archimedean, namely, there exists $u(\mathbf{x}) \in M(\mathbf{g})$ such that the set $\{\mathbf{x} \in \mathbb{R}^n | u(\mathbf{x}) \geq 0\}$ is compact. If $f(\mathbf{x})$ is strictly positive on $\mathcal{K}$, then $f(\mathbf{x})$ can be represented as

$$f(\mathbf{x}) = \sigma_0(\mathbf{x}) + \sum_{i=1}^{s} \sigma_i(\mathbf{x}) g_i(\mathbf{x}), \tag{5}$$

where $\sigma_i \in \Sigma[\mathbf{x}], 0 \leq i \leq s$.

From Theorem 2, the existence of the representation (5) provides a sufficient and necessary condition of polynomial positivity on the semi-algebraic set $\mathcal{K}$. Note that the degree bound for $\sigma_i(\mathbf{x})$ in (5) is exponential with n and $\deg(f)$. From a computational point of view, the method for finding the representation (5) has some degree of conservativeness, say, by fixing a priori much smaller degree bound D for $\sigma_i(\mathbf{x})$.

Consider the initial barrier condition again. Exploiting the fact that the barrier certificate B in the form of feed-forward neural networks with polynomial activations is polynomial, the representation (5) helps to transform the problem of checking the initial barrier condition into the following SOS problem

$$\left.\begin{array}{ll} B^*_{SOS} = & \max\limits_{\mathbf{x}} \ \lambda \\ \text{s.t.} & B_\ell(\mathbf{x}) - \lambda = \sigma_0(\mathbf{x}) + \sum_{j=1}^{m_2} \sigma_j(\mathbf{x}) p_j(\mathbf{x}) \end{array}\right\} \tag{6}$$

with $\deg(\sigma_j) \leq D$, $\sigma_j \in \Sigma[\mathbf{x}], 1 \leq j \leq m_2$.

In comparison with the optimization problem (4), the feasible set of (6) is the subset of the one of (4), which results in the optimum of (6) is the upper bound of the optimum of (4), i.e., $B^*_{POP} \leq B^*_{SOS}$.

Clearly, when the optimum $B^*_{SOS} < 0$, the optimal solution $\mathbf{x}^*_{SOS}$ can serve as the enhanced worst counterexample of the initial barrier condition, and will be passed back to guide *learner* to refine the candidate barrier certificate. If the optimum $B^*_{SOS} \geq 0$, the verification problem remains unresolved. To be complete, our approach leverages a CAD based solver to resolve the problem of checking the satisfiability of the initial barrier conditions. Note that the other three barrier conditions in Theorem 1 can be handled similarly.

Example 1 continued. For Example 1, we proceed to verify correctness of the trained candidate neural barrier certificate $\tilde{B}(\mathbf{x})$. By searching for the most suspected point via SOS programming, we obtain the enhanced worst counterexample, $\mathbf{x}^*_{SOS} = (-0.9327, 0.0954, 0.1339)$, which violates the second barrier condition. The counterexample is fed to guide the *learner* to refine the candidate barrier. After three iterations, the following truly neural barrier certificate is synthesized

$$\begin{aligned} B(\mathbf{x}) = & 0.9783x_1^2 + 0.1626x_1x_2 - 1.3742x_1x_3 - 0.2205x_1 \\ & + 0.3224x_2^2 + 1.9319x_2x_3 + 0.0828x_2 + 0.4863x_3^2 \\ & - 0.1275x_3 - 1.7020, \end{aligned}$$

which is plotted in Figure 2. It is shown that the zero level set of the barrier certificate $B(\mathbf{x})$ (the yellow surface) separates $\mathcal{X}_u$ (the red ball) from all trajectories starting from $\mathcal{X}_0$ (the green ball).

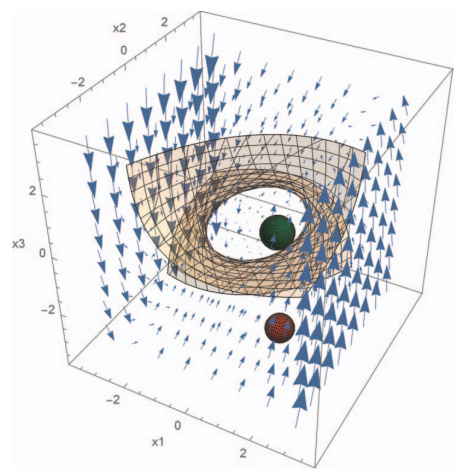

Fig. 2. Phase portrait of the system in Example 1 (Green ball: $\mathcal{X}_0$; Red ball: $\mathcal{X}_u$; Yellow surface: the zero level set of $B(\mathbf{x})$.

Alternatively, we also tried to individually apply the FIND-INSTANCE function in Mathematica to search for counterexamples of the trained candidate neural barrier certificate $\tilde{B}(\mathbf{x})$. The CAD base method finally yields a real barrier certificate with five iterations. Furthermore, the state-of-the-art CEGIS framework FOSSIL [10] manages to compute a real barrier certificate after 19 iterations. □

C. The learning Algorithm

Following the above discussions, we present an algorithm to synthesize neural barrier certificates for safety verification of hybrid systems.

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 shows the iteration scheme of our barrier certificate synthesis, which guides the experiment implementation. The procedure takes as inputs a hybrid system $\mathbf{H}$, the domain $\Psi(\ell), \ell \in L$, initial region Θ, the transition set G_e, unsafe region X_u, maximum number of iteration *MaxIter*, loss function parameters ϵ_1, ϵ_2, ϵ_3, ϵ_4, the learning rate α, and returns valid barrier certificates of a given architecture or Failure. In a pass of the iteration, the implementation process has three steps as follows.

(i) Four training datasets $(\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}, \mathcal{D}_{\ell,4})$ from $(\Theta, \Psi(\ell), G_e, X_u)$ are constructed corresponding to the barrier conditions in Theorem 1. The learner trains neural candidates and updates the weights *W* and biases *b* of the network until the empirical barrier loss converges to zero in Line (4-9).

(ii) The verifier gets the trained candidate from step (i) and identifies its validation. With the help of SOS programming, the worst counter examples in related region that violated the constraints most were yielded out, which were to be added into the respective training sets for further guiding the learner in Line (10,11,16). If the problem remains unresolved, a CAD based solver was adopted to resolve it.

(iii) If the candidate did satisfy all of the four conditions in equation (2) with the verification by a CAD based solver, we could conclude that the candidate was a real barrier certificate in Line (12-14).

This inductive loop between the learner and the verifier repeats until no counter example exists or the number of iteration *iter* reaches the maximum number of iteration *MaxIter*.

Algorithm 1 Synthesis of Barrier Certificate

Input: $\mathbf{H}$, Θ, $\Psi(\ell)$, G_e, X_u, *MaxIter*, ϵ_1, ϵ_2 ϵ_3, ϵ_4 and α
Output: $B(\mathbf{x})$ or Failure

1: $iter \leftarrow 0$;
2: $(\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}, \mathcal{D}_{\ell,4}) \leftarrow$Sampling$(\Theta, \Psi(\ell), G_e, X_u)$, Initialize NN$_\theta$
3: **while** $iter < maxIter$ **do**
4: **repeat**
5: $B_\theta(\mathbf{x}) \leftarrow NN_\theta(\mathbf{x})$
6: Compute empirical barrier loss $\widetilde{\mathcal{L}}(\theta)$
7: $W \leftarrow W + \alpha \nabla_W \widetilde{\mathcal{L}}$
8: $b \leftarrow b + \alpha \nabla_b \widetilde{\mathcal{L}}$
9: **until** convergence
10: Encode the conditions in Equation(2)
11: the worst CEX or UNSAT $\leftarrow$ SOScheck $(\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}, \mathcal{D}_{\ell,4}, B_\theta(\mathbf{x}))$
12: **if** SOScheck$(\Theta, \Psi(\ell), X_u, G_e, B_\theta(\mathbf{x}))$ == UNSAT **then**
13: CADcheck$(\Theta, \Psi(\ell), X_u, G_e, B_\theta(\mathbf{x}))$
14: Return $B_\theta(\mathbf{x})$
15: **else**
16: $(\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}, \mathcal{D}_{\ell,4}) \leftarrow (\mathcal{D}_1, \mathcal{D}_{\ell,2}, \mathcal{D}_{\ell,3}, \mathcal{D}_{\ell,4}) \cup$ the worst CEX
17: **end if**
18: **end while**

IV. Experiments

We have implemented a barrier certificate synthesis tool named *CEGIS4BC* based on Algorithm 1 using Tensorflow 2.0 platform. We apply our tool to verify a number of benchmark examples from the literature and compare with state-of-the-art CEGIS framework FOSSIL [10]. In Fossil, Satisfiability Modulo Theory (SMT) solvers Z3 and dReal are leveraged to guarantee the formal correctness of the candidate barrier certificates. Z3 uses an elegant SMT-adaptation of the CAD method, and constructs CAD only partially to explain conflicts in the Boolean SAT search. Z3 just obtains a counterexample at a time, and cannot handle non-polynomial formulae. dReal is based on the delta complete procedure, which decides SAT and weak-UNSAT of inequalities. dReal is capable of handling non-polynomial terms, but may produce spurious counterexamples, thus increase the number of iterations.

In our experiments, we ran several attempts for each example by testing different configurations of networks whose hidden layers ranged from 1 to 3 and the number of hidden neurons was chosen among $\{5, 10, 20, 30\}$. We started from simpler structure and incremented the number of the layers until the desired results were obtained. We kept the network parameters that can calculate the valid barrier certificate and reported the corresponding experimental data. Of course, we did the same work for the compared algorithm, too. Similarly, we also try to achieve each example via different polynomial activation functions with degrees ranging from 1 to 5. And we would increase the degree if we failed to obtain the desired results with the previous degree.

The following examples are given to illustrate how our tool synthesizes valid neural barrier certificates.

Example 2: Consider the following continuous dynamical system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -x_1 - x_2 + \frac{1}{3}x_1^3 \end{bmatrix}$$

with domain $\mathcal{X} = \left\{\mathbf{x} \in \mathbb{R}^2 | -3.5 \leq x_1 \leq 2, -2 \leq x_2 \leq 1\right\}$. Our goal is to check that all trajectories of the system starting from the initial set

$$\begin{aligned}\mathcal{X}_0 = \big\{\mathbf{x} \in \mathbb{R}^2 | &(x_1 - 1.5)^2 + x_2^2 \leq 0.25 \\ &\vee (x_1 \geq -1.8 \wedge x_1 \leq -1.2 \wedge x_2 \geq -0.1 \wedge x_2 \leq 0.1) \\ &\vee (x_1 \geq -1.4 \wedge x_1 \leq -1.2 \wedge x_2 \geq -0.5 \wedge x_2 \leq 0.1)\big\}\end{aligned}$$

will never reach the unsafe set

$$\begin{aligned}\mathcal{X}_u = \big\{\mathbf{x} \in \mathbb{R}^2 | &(x_1 + 1)^2 + (x_2 + 1)^2 \leq 0.16 \\ &\vee (x_1 \geq 0.4 \wedge x_1 \leq 0.6 \wedge x_2 \geq 0.1 \wedge x_2 \leq 0.5) \\ &\vee (x_1 \geq 0.4 \wedge x_1 \leq 0.8 \wedge x_2 \geq 0.1 \wedge x_2 \leq 0.3)\big\}.\end{aligned}$$

By Algorithm 1, we accomplish the goal and provide the details here. At first, the barrier certificate is parameterized as a neural network with one hidden layer of 30 neurons and polynomial activation of degree 3 approximating ReLU function. The weights among layers and biases are stochastic initialized with a standard Gaussian distribution. Network

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

parameters are updated by Adam optimizer with the learning rate at 0.05. Then we train the neural barrier certificate over the training datasets, and obtain the following candidate when the loss converges to 0:

$$\begin{aligned}\tilde{B}(\mathbf{x}) = &-2.9980x_1^5 - 274.8626x_2^5 - 0.7806x_1^4x_2 + 2.7069x_1^3x_2^2 \\ &+ 43.4094x_1x_2^4 - 13.2088x_1^2x_2^3 + 372.977x_2^4 - 34.7509x_1^3x_2 \\ &+ 12.5372x_1x_2^3 + 35.9750x_1^2x_2^2 - 16.3614x_1^2x_2 - 3.8892x_1^3 \\ &+ 66.3317x_2^3 - 28.1474x_1x_2^2 + 40.5272x_1x_2 + 9.6838x_1^2 - \\ &8.4700x_1^4 - 128.7347x_2^2 + 13.1397x_1 + 40.5822x_2 - 9.2512.\end{aligned}$$

By solving the SOS problem (6), the enhanced worst counterexample, $\mathbf{x}^*_{SOS} = (-1.7998, 0.0996)$, of the initial barrier condition is computed. Similarly, we can obtain the enhanced worst counterexample $(0.1992, 0.2716)$, with respect to the second barrier condition. As shown in Figure 3, the zero level set of the candidate barrier certificate $\tilde{B}(\mathbf{x})$ (the blue solid line) intersects with the initial set $\mathcal{X}_0$ (the green circle), which means the candidate violates the initial barrier condition. Therefore, the candidate is not a real barrier certificate.

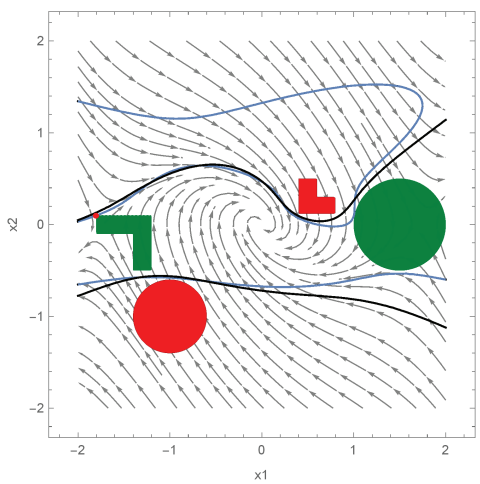

Fig. 3. Phase portrait of the system in Example 2 (Green region: $\mathcal{X}_0$; Red region: $\mathcal{X}_u$; Blue solid line: the zero level set of $\tilde{B}(\mathbf{x})$; Black solid line: the zero level set of $B(\mathbf{x})$).

The counterexamples mentioned above are then passed back to retrain the candidate. Finally, we obtain the following barrier certificate

$$\begin{aligned}B(\mathbf{x}) = &-5.5751x_1^5 - 169.3943x_2^5 + 2.7537x_1^4x_2 + 45.4087x_1^2x_2^3 \\ &+ 0.7833x_1^3x_2^2 - 3.5398x_1x_2^4 - 35.0285x_1^3x_2 + 44.2641x_1^2x_2^2 \\ &+ 24.5623x_1x_2^3 - 10.8632x_1^2x_2 - 18.6770x_1^4 + 277.5585x_2^4 \\ &- 23.7842x_1x_2^2 + 50.0719x_2^3 - 14.1033x_1^3 + 72.0145x_1x_2 + \\ &14.4348x_1^2 - 85.8866x_2^2 + 22.0165x_1 + 40.6922x_2 - 15.1048,\end{aligned}$$

for which the SOS programs, associated to the problems of checking the barrier certificate conditions in Theorem 1, all have nonnegative optimums. To be rigorous, the candidate certificate is then formally verified via the Mathematica CAD solver. As shown in Figure 3, the zero level set of the valid barrier certificate $B(\mathbf{x})$ (the black solid line) separates $\mathcal{X}_u$ (the red circle) from all trajectories starting from $\mathcal{X}_0$ (the green circle). Thus, the $B(\mathbf{x})$ is a valid barrier certificate and the safety of the above system is verified. □

Example 3: Consider the following non-polynomial dynamical system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} e^{-x_1} + x_2 - 1 \\ -\sin^2 x_1 \end{bmatrix}$$

with domain $\mathcal{X} = \left\{\mathbf{x} \in \mathbb{R}^2 | -2 \le x_1 \le 2, -2 \le x_2 \le 2\right\}$. We aim to check that all trajectories of the system starting from the initial set

$$\mathcal{X}_0 = \left\{\mathbf{x} \in \mathbb{R}^2 | (x_1 + 0.5)^2 + (x_2 - 0.5)^2 \le 0.16\right\},$$

will never reach the unsafe set

$$\mathcal{X}_u = \left\{\mathbf{x} \in \mathbb{R}^2 | (x_1 - 0.7)^2 + (x_2 + 0.7)^2 \le 0.09\right\}.$$

Using a neural network with one hidden layer consisting of 30 neurons and polynomial activation of degree 3 approximating ReLU function, a candidate barrier certificate is generated

$$\begin{aligned}\tilde{B}(\mathbf{x}) = &- 5.0936 + 5.1079x_1 - 3.4322x_2 + 1.5417x_1^2 \\ &+ 4.7019x_2^2 - 2.9924x_1x_2 + 4.7696x_2^3 - 0.8067x_1^3 \\ &+ 1.6387x_1^2x_2 - 2.1324x_1x_2^2.\end{aligned}$$

We proceed to verify the correctness of the trained candidate barrier certificate. The FINDINSTANCE function in Mathematica returns several counterexamples (such as the red point $(0.1256, 0.9461)$ in Figure 4), within the zero level set of the candidate barrier certificate $\tilde{B}(\mathbf{x})$ (the blue solid line in Figure 4), make the value of the lie derivative less than zero. It means that the candidate violates the second barrier condition in Theorem 1, thus the candidate is not a true barrier certificate.

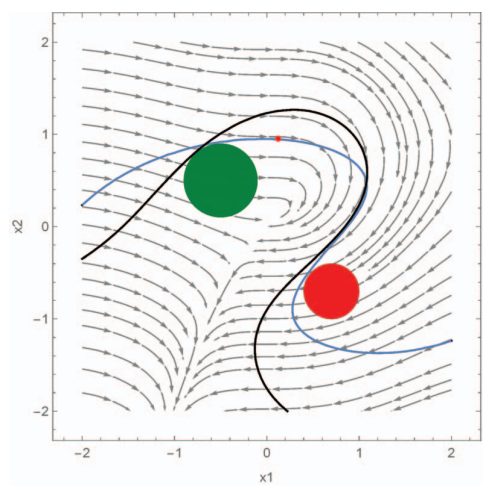

Fig. 4. Phase portrait of the system in Example 3 (Green region: $\mathcal{X}_0$; Red region: $\mathcal{X}_u$; Blue solid line: the zero level set of $\tilde{B}(\mathbf{x})$; Black solid line: the zero level set of $B(\mathbf{x})$).

After two iterations, we finally obtain the following true barrier certificate

$$\begin{aligned}B(\mathbf{x}) = &- 7.3465 + 5.1122x_1 - 6.3616x_2 - 7.5563x_1x_2 \\ &+ 5.3710x_2^2 - 0.8786x_1^3 + 4.9492x_1^2x_2 + 4.9218x_1^2 \\ &- 1.0868x_1x_2^2 + 3.7670x_2^3.\end{aligned}$$

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

As shown in Figure 4, the zero level set of the barrier certificate $B(\mathbf{x})$ (the black solid line) separates $\mathcal{X}_u$ (the red circle) from all trajectories starting from $\mathcal{X}_0$ (the green circle). Thus, the safety of the above system is verified. □

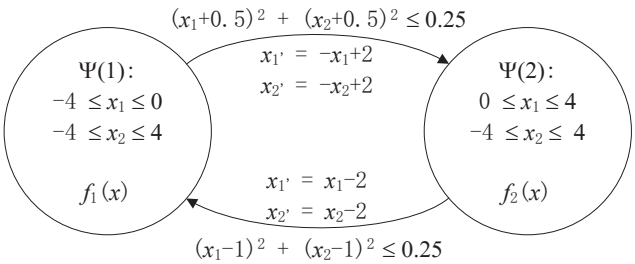

Fig. 5. The hybrid system in Example 3.

Example 4: Consider the following hybrid system in Figure 5 where

$$f_1(x) : \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_1 - x_1x_2 \\ -x_2 + x_1x_2 \end{bmatrix}$$

$$f_2(x) : \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_1 + x_1^2x_2 \\ x_2 + x_1x_2 \end{bmatrix}.$$

Our goal is to check that all trajectories of the system starting from the initial set Θ in location ℓ_1:

$$\Theta = \{\mathbf{x} \in \mathbb{R}^2 | -2 \le x_1, x_2 \le -1\},$$

will never reach the unsafe set X_u in location ℓ_2:

$$X_u = \{\mathbf{x} \in \mathbb{R}^2 | 0 \le x_1 \le 1, -2 \le x_2 \le -1\}.$$

We first parameterize the neural barrier certificate for each location as a neural network with one hidden layer of 20 neurons and polynomial activation of degree 2 approximating ReLU function. Network parameters are updated by Adam optimizer with the learning rate at 0.1. Then we obtain the following candidate barrier certificates when the loss converges to 0:

$$\begin{aligned} B_1(x) = &-0.0008x_1^2 - 0.0114x_1x_2 + 6.9906x_1 - 0.0082x_2^2 \\ &- 0.0059x_2 - 3.8951, \\ B_2(x) = &-0.0018x_1^2 - 0.0184x_1x_2 + 1.6431x_1 - 0.0097x_2^2 \\ &- 11.7019x_2 + 8.9931. \end{aligned}$$

What's more, the validity of the two candidates are checked via the Mathematica CAD solver. Thus, the $B_1(\mathbf{x})$ and $B_2(\mathbf{x})$ are valid barrier certificates and the safety of the hybrid system is verified. □

Table 1 shows the performance evaluation of our *CEGIS4BC* and FOSSIL on a set of benchmarks. The origins of these examples are provided in the first column of the table. dim denotes the input dimension of the dynamic system, and $\mathcal{N}$ means the structure of neural networks used for training candidates, which shows the number of hidden layers and the neurons in each hidden layer. d denotes the maximal degree of the polynomials in the vector fields. t_l and t_v is the time in seconds of learning and verification, respectively. $Iters$ represents the number of CEGIS iterations. The notation $-$ stands for that the corresponding method fails to synthesize a valid polynomial barrier certificate of degree less than 6.

According to Table 1, our tool *CEGIS4BC* can synthesize real barrier certificates for all the listed 17 examples while the FOSSIL tool cannot deal with C8, C9, and H2. For C2, C3, C6 and C7, our *CEGIS4BC* can yield simpler polynomial barrier certificates, which is barrier certificates with lower degree. Our *CEGIS4BC* seems to provide a better solving capability. The reason is that the approximate polynomials of ReLU activation inherit intrinsic properties from ReLU function to some extent, thus facilitates the training of neural barrier certificates.

Considering the number of CEGIS iterations, our *CEGIS4BC* performs better than the FOSSIL tool. The average number of CEGIS iterations the FOSSIL tool treated for each example is more than 5, which is about 3 times as many as our *CEGIS4BC*. Especially, for C6, our *CEGIS4BC* manages to handle it with 3 iterations, while the FOSSIL tool needs 19 iterations. The strategies employed in our *CEGIS4BC* that search for informative counterexamples do contribute to reducing the number of CEGIS iterations.

Moreover, to evaluate the performance of our two strategies: the CAD method and the SOS programming, on the number of CEGIS iterations, we also tried to apply the CAD solver individually to search for counterexamples of the candidate barrier certificates. Especially, for C2, C4, C5 and C6, the CAD method respectively spends 6, 4, 4 and 5 iterations, while the SOS programming method just needs 2, 3, 2, and 3 iterations, respectively. These two strategies can yield informative counterexamples with respect to barrier conditions from different perspectives. Alternatively, the FOSSIL tool manages to compute real barrier certificates with 10, 8, 6, and 19 iterations, respectively.

In addition, consider the efficiency of our tool and FOSSIL in the aspect of time spent in getting a valid barrier certificate for the listed examples. On average, our *CEGIS4BC* needs about 10 seconds to synthesize a valid barrier certificate while the FOSSIL tool takes more than 30 seconds, which is 3 times as our method. Despite that the neural structure used in both tools is the same, it is obvious that our *CEGIS4BC* is more efficient to synthesize a valid barrier certificate.

V. Related Work

Our work on synthesizing neural barrier certificates for verifying the safety property of nonlinear systems is mainly related to two categories of research, i.e. safe polynomial barrier certificate generation and formal verification of nonlinear systems with Neural barrier certificate. There has been considerable research conducted in these areas because of the applications in safety-critical systems in recent years.

Polynomial barrier certificate generation. Safety verification based on barrier certificates was first proposed by Prajna et al. in [14], [27], where the theory of Putinar's Positivstellensatz is exploited to form an SOS program of the barrier certificate. Kapinski et al. [28], [29] introduced a Lyapunov function

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ALGORITHM PERFORMANCE ON BENCHMARKS

Ex	dim	$\mathcal{N}$	*CEGIS4BC*				Fossil [10]			
			d	t_l	t_v	$Iters$	d	t_l	t_v	$Iters$
C1 [19]	2	2 - 30 - 1	3	2.04	0.86	1	3	4.16	0.15	2
C2 [20]	2	2 - 30 - 1	2	1.29	0.5	2	5	148.77	0.28	10
C3 [21]	2	2 - 30 - 1	3	0.67	0.01	1	5	123	0.47	8
C4 [21]	2	2 - 30 - 1	4	14.36	0.12	3	4	63.29	0.27	8
C5 [22]	3	3 - 30 - 1	3	9.81	0.05	2	3	26.42	0.40	6
C6 [23]	3	3 - 30 - 1	2	5.07	0.21	3	3	53.16	9.77	19
C7 [24]	4	4 - 30 - 1	3	57.66	326.60	3	4	82.53	307.72	3
C8 [25]	2	2 - 30 - 1	3	2.14	0.12	3	-	-	-	-
C9 [10]	2	2 - 30 - 30 - 1	6	10.17	0.78	4	-	-	-	-
C10 [10]	2	2 - 30 - 1	1	0.52	0.003	1	1	13.8	0.04	2
C11 [10]	2	2 - 30 - 1	3	3.85	0.02	2	3	15.9	0.07	2
C12 [10]	3	3 - 30 - 1	3	2.41	2.91	2	3	21.6	4.05	5
C13 [10]	4	4 - 10 - 1	1	3.48	0.15	1	1	29.31	0.07	1
C14 [10]	6	6 - 10 - 1	1	2.34	2.39	1	1	8.94	0.78	1
C15 [10]	8	8 - 10 - 1	1	0.64	7.26	1	1	8.02	5.71	1
H1 [10]	2	2 - 5 - 1	2	2.80	0.05	1	2	5.84	0.18	3
H2 [26]	2	2 - 20 - 1	2	2.02	0.003	1	-	-	-	-

typed barrier certificate. For semi-algebraic hybrid systems, Kong et al. [30], [31] proposed an exponential condition to generate barrier certificates. Sloth et al. [32] proposed a new barrier certificate for a special class of hybrid systems that consists of several interconnected subsystems. Dai et al. [33] studied how to relax the condition of barrier certificates in a general way while retaining the convexity. Tuncali et al. [34] combined a simulation-guided technique with a validation process to identify barrier certificates. Zeng et al. [26] presented the Darboux-type barrier certificates that define an algebraic curve, not allowing any trajectories of the system to leave it once they touch it. For hybrid systems containing elementary functions, Liu et al. [35] proposed a symbolic abstraction approach that replaces all non-polynomial terms with newly introduced variables so that they can be verified using the well-established verification techniques of polynomial hybrid systems. All the barrier certificates mentioned above are in the form of polynomials.

To our best knowledge, the seminal work introducing neural networks as barrier certificates is proposed by Zhao et al [8]. They exploit neural networks with Bent-ReLU activation functions to verify dynamical systems. After obtaining candidate networks, they first use piece-wise lines to over-approximate the Bent-ReLU function, and then invoke the non-linear SMT solver isat3 to check the barrier certificate conditions. To get an over-approximation of Bent-ReLU function precise enough to enable correct SMT judgement, the intervals of piece-wise lines should be carefully determined. They do not give an automatic interval determination method in the paper. In fact, to avoid the accumulation of approximation errors, in their experiments, all networks are of only one hidden layer where the number of neurons ranges from 5 to 20. Peruffo et al. [11] presented a CEGIS-based technique that used counterexamples to speed up the construction of networks, serving as the barrier certificates for safety verification of hybrid systems. They used polynomials as activation functions of the network and directly resorted to the non-linear SMT solver dReal and Z3 to check the barrier certificate conditions. Experiments showed their method was faster and required fewer data points in barrier certificate synthesis than Zhao's method.

Different from them, we adopt a special feedforward neural network structure that consists of one or two hidden layers and uses the polynomial activation functions to synthesize candidate barrier certificates, and employ a SOS based technique to identify the real ones.

Neural barrier certificate Verification. Synthesized barrier certificates are subject to further verification, which is related to formal verification of neural networks. The problem of neural network verification is NP-hard [36] and lots of research focuses on it [37]–[40]. Existing methods for verifying neural network are based on abstract domain [41], abstract interpretation [37], input refinement [36], interval arithmetic [39], linear approximations [40] and mixed integer programming [38], [42], [43]. The previous MIP-based verification method is used to verify the output of neural network neurons on a given input region, which is a problem about mixed integer linear programming optimization. In this work, we adopt a SOS-based method to check barrier certificate conditions. For those conditions determining neural network output, we encode them as several polynomial optimization problems. Moreover, for the condition concerning non-polynomial, we convert them into polynomial problems by piece-wise approximation.

VI. CONCLUSION

In this paper, we have presented an approach to synthesize neural barrier certificates efficiently for verifying the safety of hybrid system. Our synthesis employs a CEGIS procedure with two stages: neural candidate barrier training and formal verification for the candidate. The Learner constructs a candidate represented by neural network which exploits approximate polynomials of ReLU activation function. The Verifier adopts SOS programming for speeding up identifying real barrier certificate from the candidate one, by finding the point that is the most likely to violate the barrier conditions.

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.

We have implemented the tool and evaluated its performance over a number of benchmark examples. The experimental result shows that our tool is more effective and efficient than the classical iterative method.

REFERENCES

[1] S. Gulwani and A. Tiwari, "Constraint-based approach for analysis of hybrid systems," in *Computer Aided Verification, 20th International Conference, CAV 2008, Proceedings*. Springer, 2008, pp. 190–203.

[2] T. Sturm and A. Tiwari, "Verification and synthesis using real quantifier elimination," in *Proceeding of Symbolic and Algebraic Computation, International Symposium, ISSAC 2011*, 07 2011, pp. 329–336.

[3] A. Djaballah, A. Chapoutot, M. Kieffer, and O. Bouissou, "Construction of parametric barrier functions for dynamical systems using interval analysis," *Automatica*, pp. 287–296, 2017.

[4] Z. Yang, M. Wu, and W. Lin, "An efficient framework for barrier certificate generation of uncertain nonlinear hybrid systems," *Nonlinear Analysis: Hybrid Systems*, p. 100837, 2020.

[5] X. Chen, C. Peng, W. Lin, Z. Yang, Y. Zhang, and X. Li, "A novel approach for solving the bmi problem in barrier certificates generation," in *Computer Aided Verification*, 2020, pp. 582–603.

[6] Q. Wang, M. Chen, B. Xue, N. Zhan, and J. Katoen, "Synthesizing invariant barrier certificates via difference-of-convex programming," in *Computer Aided Verification - 33rd International Conference, CAV 2021, Proceedings*, vol. 12759. Springer, 2021, pp. 443–466.

[7] Y. Lu and J. Lu, "A universal approximation theorem of deep neural networks for expressing distributions," *arXiv preprint arXiv:2004.08867*, 2020.

[8] H. Zhao, X. Zeng, T. Chen, and Z. Liu, "Synthesizing barrier certificates using neural networks," in *HSCC '20: 23rd ACM International Conference on Hybrid Systems: Computation and Control*. ACM, 2020, pp. 25:1–25:11.

[9] Q. Zhao, X. Chen, Y. Zhang, M. Sha, Z. Yang, W. Lin, E. Tang, Q. Chen, and X. Li, "Synthesizing relu neural networks with two hidden layers as barrier certificates for hybrid systems," in *HSCC '21: 24th ACM International Conference on Hybrid Systems: Computation and Control*, 2021, pp. 17:1–17:11.

[10] A. Abate, D. Ahmed, A. Edwards, M. Giacobbe, and A. Peruffo, "Fossil: A software tool for the formal synthesis of lyapunov functions and barrier certificates using neural networks," in *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control*, 2021.

[11] A. Peruffo, D. Ahmed, and A. Abate, "Automated and formal synthesis of neural barrier certificates for dynamical models," in *Tools and Algorithms for the Construction and Analysis of Systems - 27th International Conference, TACAS 2021, Proceedings*, 2021, pp. 370–388.

[12] R. Alur, C. Courcoubetis, N. Halbwachs, T. A. Henzinger, P.-H. Ho, X. Nicollin, A. Olivero, J. Sifakis, and S. Yovine, "The algorithmic analysis of hybrid systems," *Theoretical Computer Science*, vol. 138, no. 1, pp. 3–34, 1995.

[13] Z. Yang, M. Wu, and W. Lin, "Exact verification of hybrid systems based on bilinear SOS representation," *ACM Transactions on Embedded Computing Systems*, vol. 14, no. 1, pp. 1–19, 2015.

[14] S. Prajna, A. Jadbabaie, and G. Pappas, "A framework for worst-case and stochastic safety verification using barrier certificates," *Automatic Control, IEEE Transactions on*, vol. 52, pp. 1415 – 1428, 09 2007.

[15] C. W. Brown, "Improved projection for cylindrical algebraic decomposition," *J. Symb. Comput.*, vol. 32, no. 5, pp. 447–465, 2001.

[16] G. E. Collins and H. Hong, "Partial cylindrical algebraic decomposition for quantifier elimination," *J. Symb. Comput.*, vol. 12, no. 3, pp. 299–328, 1991.

[17] A. W. Strzebonski, "Cylindrical algebraic decomposition using validated numerics," *J. Symb. Comput.*, vol. 41, no. 9, pp. 1021–1038, 2006.

[18] M. Putinar, "Positive polynomials on compact semi-algebraic sets," *Indiana University Mathematics Journal*, vol. 42, pp. 968–984, 1993.

[19] X. Chen, E. Ábrahám, and S. Sankaranarayanan, "Taylor model flowpipe construction for non-linear hybrid systems," in *Proceedings of the 33rd IEEE Real-Time Systems Symposium, RTSS 2012*. IEEE Computer Society, 2012, pp. 183–192.

[20] A. Sogokon, K. Ghorbal, and T. T. Johnson, "Non-linear Continuous Systems for Safety Verification (Benchmark Proposal)," in *ARCH@CPSWeek 2016 - 3rd International Workshop on Applied Verification for Continuous and Hybrid Systems*, Apr. 2016, pp. 42–51.

[21] O. Bouissou, A. Chapoutot, A. Djaballah, and M. Kieffer, "Computation of parametric barrier functions for dynamical systems using interval analysis," in *53rd IEEE Conference on Decision and Control, CDC 2014*, 2014, pp. 753–758.

[22] M. A. B. Sassi and S. Sankaranarayanan, "Stabilization of polynomial dynamical systems using linear programming based on bernstein polynomials," *arXiv preprint arXiv:1501.04578*, 2015.

[23] S. Ratschan and Z. She, "Providing a basin of attraction to a target region of polynomial systems by computation of lyapunov-like functions," *SIAM J. Control. Optim.*, vol. 48, no. 7, pp. 4377–4394, 2010.

[24] S. Sankaranarayanan, X. Chen, and E. Ábrahám, "Lyapunov function synthesis using handelman representations," in *9th IFAC Symposium on Nonlinear Control Systems, NOLCOS 2013*, 2013, pp. 576–581.

[25] S. Ratschan and Z. She, "Constraints for continuous reachability in the verification of hybrid systems," in *Artificial Intelligence and Symbolic Computation, 8th International Conference, AISC 2006, Proceedings*, 2006, pp. 196–210.

[26] X. Zeng, W. Lin, Z. Yang, X. Chen, and L. Wang, "Darboux-type barrier certificates for safety verification of nonlinear hybrid systems," in *Proceedings of the 13th International Conference on Embedded Software*. Association for Computing Machinery, 2016.

[27] S. Prajna and A. Jadbabaie, "Safety verification of hybrid systems using barrier certificates," in *In Hybrid Systems: Computation and Control*, 2004, pp. 477–492.

[28] J. Kapinski and J. Deshmukh, *Discovering Forward Invariant Sets for Nonlinear Dynamical Systems*, 2015, pp. 259–264.

[29] J. Kapinski, J. V. Deshmukh, S. Sankaranarayanan, and N. Arechiga, "Simulation-guided lyapunov analysis for hybrid dynamical systems," in *Proceedings of the 17th International Conference on Hybrid Systems: Computation and Control*, 2014, p. 133142.

[30] H. Kong, F. He, X. Song, W. N.N.Hung, and M. Gu, "Exponential-condition-based barrier certificate generation for safety verification of hybrid systems," in *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, 2013, pp. 242–257.

[31] H. Kong, X. Song, D. Han, M. Gu, and J. Sun, "A new barrier certificate for safety verification of hybrid systems," *The Computer Journal*, vol. 57, pp. 1033–1045, 07 2014.

[32] C. Sloth, G. Pappas, and R. Wisniewski, "Compositional safety analysis using barrier certificates," *HSCC'12 - Proceedings of the 15th ACM International Conference on Hybrid Systems: Computation and Control*, pp. 15–24, 04 2012.

[33] L. Dai, T. Gan, B. Xia, and N. Zhan, "Barrier certificates revisited," *Journal of Symbolic Computation*, vol. 80, pp. 62–86, 2017.

[34] C. E. Tuncali, J. Kapinski, H. Ito, and J. V. Deshmukh, "Reasoning about safety of learning-enabled components in autonomous cyber-physical systems," in *Proceedings of the 55th Annual Design Automation Conference*. Association for Computing Machinery, 2018.

[35] J. Liu, N. Zhan, H. Zhao, and L. Zou, "Abstraction of elementary hybrid systems by variable transformation," 03 2014.

[36] G. Katz, C. Barrett, D. Dill, K. Julian, and M. Kochenderfer, "Reluplex: An efficient smt solver for verifying deep neural networks," 02 2017.

[37] G. Singh, T. Gehr, M. Mirman, M. Püschel, and M. Vechev, "Fast and effective robustness certification," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. Curran Associates Inc., 2018, p. 1082510836.

[38] V. Tjeng and R. Tedrake, "Verifying neural networks with mixed integer programming," 11 2017.

[39] S. Wang, K. Pei, J. Whitehouse, J. Yang, and S. Jana, "Formal security analysis of neural networks using symbolic intervals," 07 2018.

[40] E. Wong and Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80. PMLR, 2018, pp. 5286–5295.

[41] G. Singh, T. Gehr, M. Puschel, and M. Vechev, "An abstract domain for certifying neural networks," vol. 3, no. POPL, 2019.

[42] S. Dutta, S. Jha, S. Sankaranarayanan, and A. Tiwari, *Output Range Analysis for Deep Feedforward Neural Networks*, 2018, pp. 121–138.

[43] V. Tjeng, K. Xiao, and R. Tedrake, "Evaluating robustness of neural networks with mixed integer programming," Ph.D. dissertation, 2017.

Authorized licensed use limited to: George Mason University. Downloaded on January 25,2026 at 00:17:16 UTC from IEEE Xplore. Restrictions apply.